

TIN 三角网生成

1.TIN 三角网

1.1 定义

TIN(Triangulated Irregular Network , 不规则三角网)是一种利用不规则分布的离散采样点, 通过三角剖分构建三角形网络, 并以三角形面片作为基本单元来表达连续地形表面的数字地形模型。TIN 是以一系列具有平面坐标和高程属性的离散点为基础, 通过三角剖分方法将这些点连接成互不重叠、空间连续的三角形面片集合, 从而形成对地形表面的分片线性逼近。在 TIN 模型中, 每个三角形由三个不共线的高程点确定, 因此每一个三角形面片都可以被视为一个局部平面。整个地形表面则由大量相邻三角形面片拼接而成。

TIN 的基础数据是离散采样点。每一个采样点通常包含平面坐标和高程信息, 可表示为: $P_i(x_i, y_i, z_i)$, 其中, x_i 和 y_i 表示点在平面坐标系中的位置, z_i 表示该点对应的高程或属性值。采样点的空间分布并不要求规则排列, 这是 TIN 与 DEM(Digital Elevation Model , 数字高程模型) 的重要区别之一。

三角剖分是 TIN 构建的核心过程。其基本思想是在给定离散点集的基础上, 将研究区域划分为一系列相邻且互不重叠的三角形。设空间点集为: $P = \{P_1, P_2, \dots, P_n\}$ 。三角剖分的目标是将该点集连接成三角形集合: $T = \{T_1, T_2, \dots, T_m\}$, 其中, 每一个三角形 T_j 由三个采样点构成, 且任意两个三角形之间只能在公共顶点或公共边处相接, 不能发生交叉或重叠。

三角形面片是 TIN 模型的基本空间单元。由于三个不共线点能够唯一确定一个平面, 因此每个三角形面片可以表示为一个局部平面: $z = ax + by + c$, 其中, a 、 b 、 c 为由三角形三个顶点坐标确定的平面参数。

TIN 存在拓扑结构, 通常包括点与边之间的关联关系、边与三角形之间的关联关系和三角形与相邻三角形之间的邻接关系。拓扑关系是进行三角形定位、三维可视化等的重要基础。

因此, TIN 模型实质上是将复杂连续地形表面划分为大量局部平面, 并通过这些局部平面的组合实现对真实地形的近似表达。

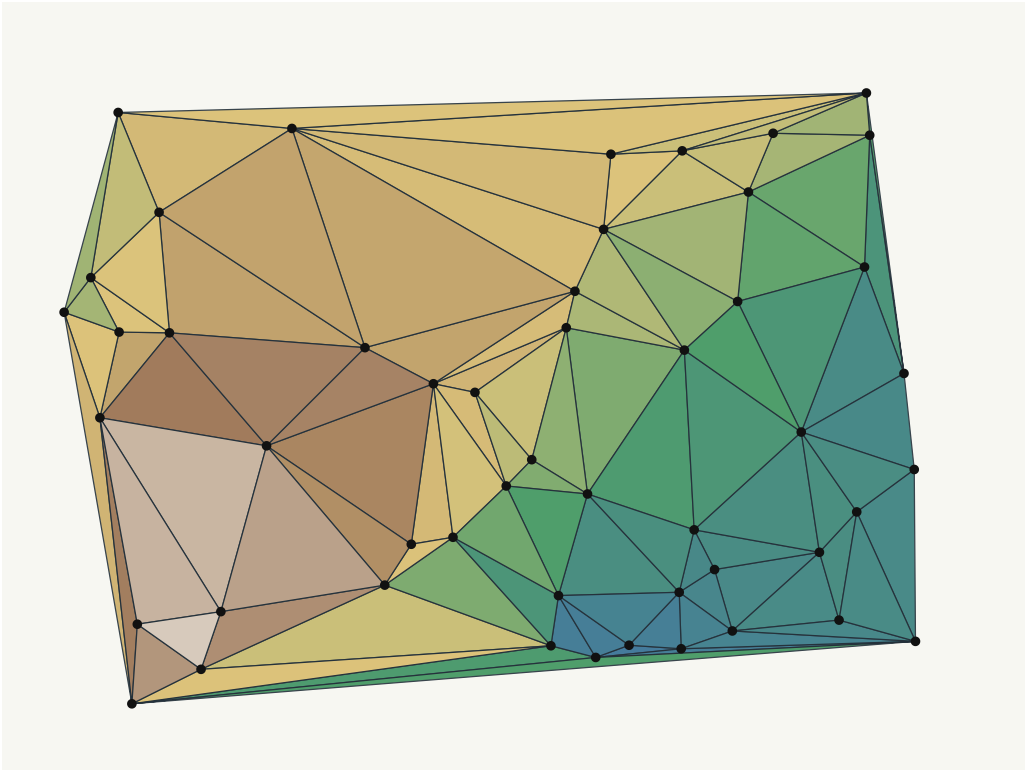


图1.1 TIN 三角网项目示意图

1.2 构建原理

TIN 的构建主要依赖于三角剖分算法，其中最常用的是 *Delaunay* 三角剖分。*Delaunay* 三角剖分具有空圆性、合法性和优化性。

Delaunay 三角剖分基本准则为对于三角网中的任意一个三角形，其外接圆内部不应包含其他采样点，即空圆准则。通过 *Voronoi* 图与 *Delaunay* 三角网之间的对偶关系，对于给定的平面离散点集，*Voronoi* 图将平面划分为若干最近邻区域，每一个区域中的点到其对应采样点的距离不大于到其他采样点的距离。当三个采样点的 *Voronoi* 区域交于同一个顶点时，该顶点到这三个采样点的距离相等，因此该点即为由这三个采样点构成三角形的外接圆圆心。若该外接圆内部存在另一个采样点，则该圆心到此采样点的距离将小于到三个三角形顶点的距离，这与 *Voronoi* 区域最近邻划分的定义相矛盾。因此，由 *Voronoi* 图对偶构造得到的三角形必然满足空圆准则，从而说明 *Delaunay* 三角剖分在几何构造上具有空圆性。

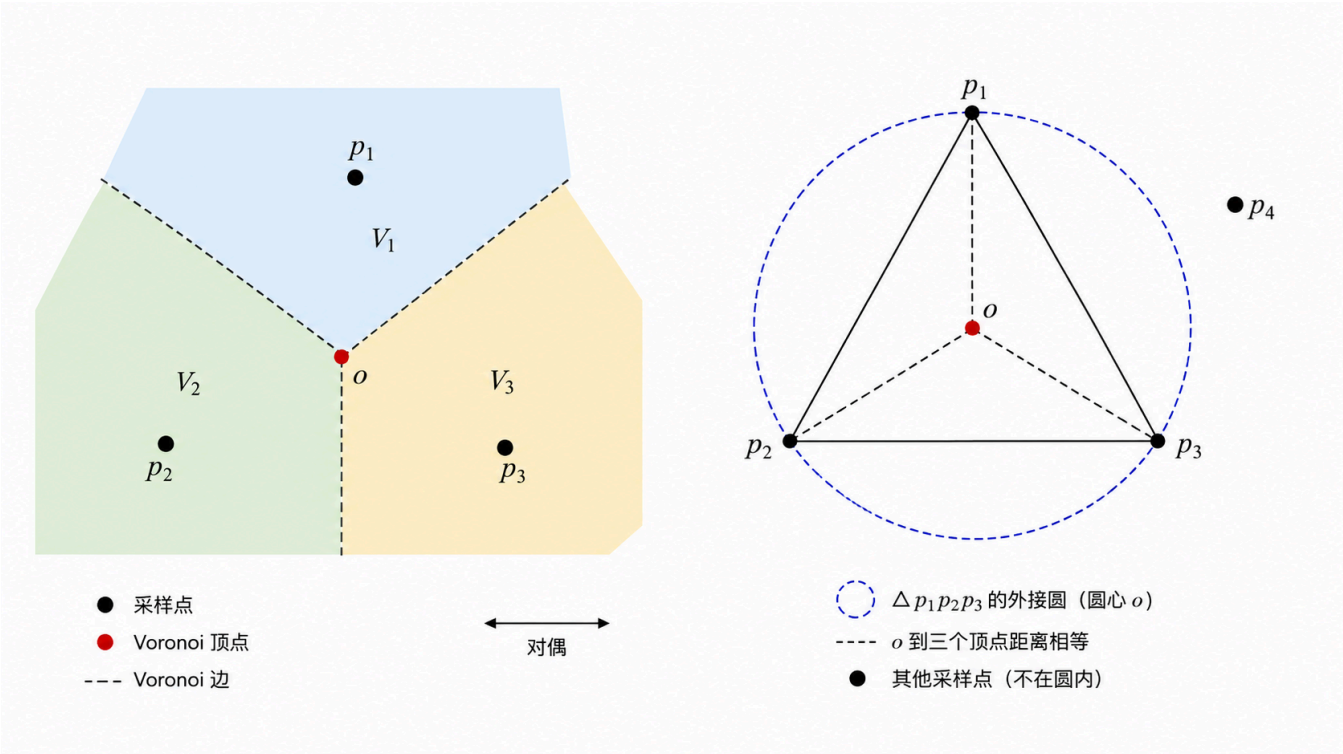


图1.2 空圆准则

Delaunay 三角剖分首先必须是一个合法的平面三角剖分，即其边不发生相互交叉，三角形之间不重叠，并且能够完整覆盖点集凸包内部区域。该性质同样可以从 *Voronoi* 图的平面划分特征得到说明。*Voronoi* 图是对平面的连续区域划分，不同 *Voronoi* 区域之间只在公共边界处相邻，而不会发生区域交叉。*Delaunay* 三角网作为 *Voronoi* 图的对偶结构，通过连接相邻 *Voronoi* 区域所对应的采样点形成边，并由三个相邻区域的公共顶点对应形成三角形。因此，*Delaunay* 边继承了 *Voronoi* 图的平面邻接关系，不会产生相互交叉。在一般位置条件下，即不存在四点共圆等退化情形时，该对偶结构能够将点集凸包内部划分为一系列互不重叠、相互邻接的三角形单元。因此，*Delaunay* 三角剖分不仅满足空圆准则，而且构成一个拓扑关系明确、空间覆盖完整的合法三角网。

Delaunay 三角剖分的重要几何优势在于其具有最大化最小角的性质，即在所有可能的三角剖分中，*Delaunay* 三角剖分倾向于使三角形的最小内角尽可能大，从而减少狭长三角形的出现。该性质可以通过局部边交换加以理解。对于由四个点构成的凸四边形，其内部通常存在两种可能的对角线剖分方式。如果当前对角线对应的某个三角形外接圆内部包含第四个点，则说明该局部剖分不满足 *Delaunay* 条件。此时删除原对角线并连接另一条对角线，可以消除局部空圆准则的违背，并提高该局部区域中三角形角度分布的均衡性。由于狭长三角形往往会降低空间插值精度，并导致坡度、坡向和等高线生成结果不稳定，因此，*Delaunay* 三角剖分通过改善三角形形状，提高了 TIN 模型在地形表达、地形分析和三维建模中的稳定性与可靠性。由此可见，*Delaunay* 三角剖分不仅在拓扑结构上是合法的，在几何性质上也具有较优的三角网质量。

1.3 数学本质

TIN 的数学本质是对连续地形表面的分片线性逼近。

真实地形表面可以看作一个连续函数： $z = f(x, y)$ 。但在实际测量中，只能获得有限数量的离散采样点。TIN 通过三角剖分将整个研究区域划分为若干三角形单元，并在每个三角形单元内部用一个线性函数近似地形表面： $z = f_j(x, y) = a_jx + b_jy + c_j$ ，其中， $f_j(x, y)$ 表示第 j 个三角形内部的局部地形函数。整个 TIN 表面可理解为由多个局部线性函数共同组成的分片函数： $f(x, y) = f_j(x, y), (x, y) \in T_j$ 。

TIN 并不是对地表进行整体单一函数拟合，而是采用局部平面拼接的方式描述复杂地形。

2.流程分析

本次实验采用 Bowyer-Watsonz 逐点插入法。

Bowyer-Watson 法是一种构造 Delaunay 三角剖分的经典逐点插入算法。其基本输入为平面离散点集，输出为满足 Delaunay 空圆准则的三角形链表。该方法的核心思想是：先构造一个覆盖全部采样点的初始三角网，然后逐个插入采样点，并在每次插入后仅对受影响的局部区域进行删除与重构，从而逐步得到满足空圆准则的三角剖分结果。

首先，进行采样点逐点插入。根据待剖分点集构造一个足够大的“超级三角形”，使所有采样点均位于该三角形内部。超级三角形并不属于最终剖分结果，而是作为算法初始化阶段的辅助结构，用于提供一个封闭的初始三角网。在算法开始时，三角形链表中仅包含该超级三角形；随后，待剖分点将按照一定顺序依次插入到当前三角网中。

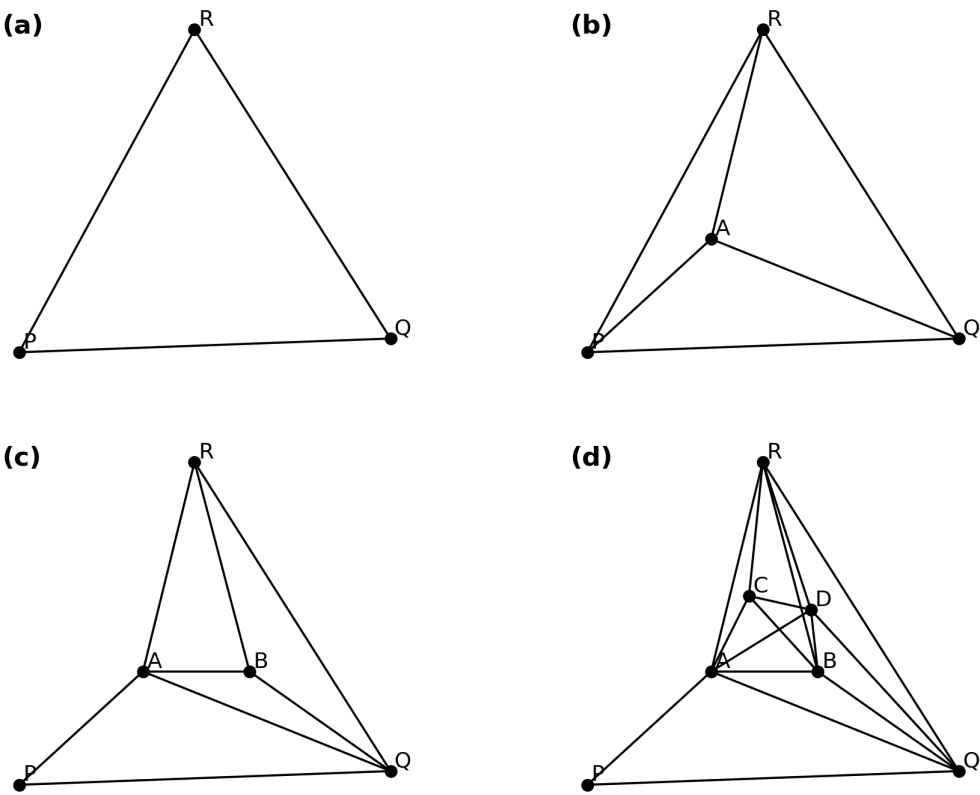


图2.1 采样点逐点插入

其次，删除不满足空圆准则的三角形。在每一次插入新点时，算法遍历当前三角形链表，并计算各三角形的外接圆。若新插入点位于某些三角形的外接圆内部，则说明这些三角形在加入新点后不再满足 Delaunay 空圆准则，因此被称为“坏三角形”。这些坏三角形需要从当前三角形链表中删除。此步骤体现了 Delaunay 三角剖分的核心判据，即任意三角形的外接圆内部不应包含其他采样点。

然后，局部空腔重构。算法将所有坏三角形的边加入边缓存，并删除其中重复出现的边。重复边通常是相邻坏三角形之间的内部公共边，不属于局部空腔的外边界；而删除重复边后保留下来的边，则构成受新点影响区域的局部空腔边界。随后，将新插入点分别与空腔边界上的每一条边相连，生成一组三角形，并将这些新三角形重新加入三角形链表。

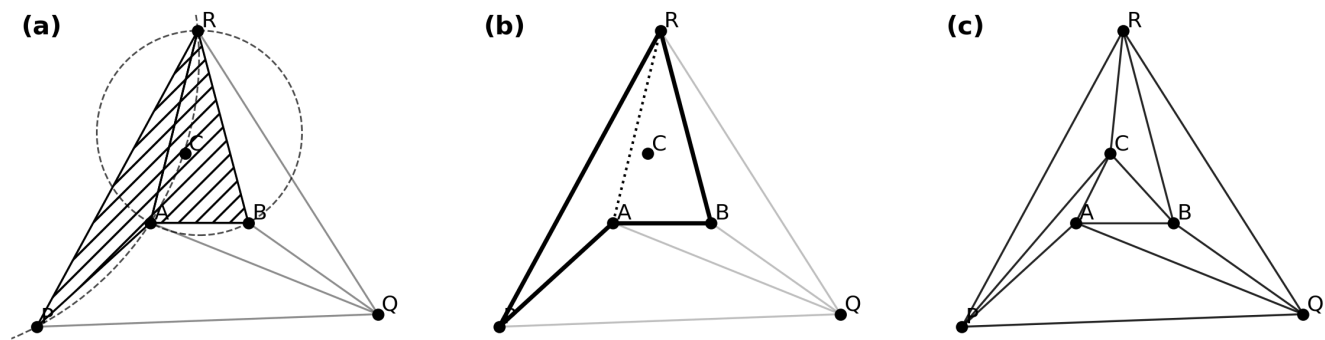


图2.2 三角形的删除和局部空腔重构

最后，删除超级三角形。上述过程中，每一次点插入都只更新局部三角网，而不需要重新构造整个剖分区域。该机制提高了算法效率，同时保证三角网在逐步更新过程中不断趋向满足 *Delaunay* 空圆准则。所有采样点插入完成后，算法删除所有与超级三角形顶点相关联的三角形，因为这些三角形只是由辅助边界产生的临时结构，并不属于原始点集的真实剖分结果。经过该清理步骤后，剩余三角形即构成最终的 *Delaunay* 三角剖分结果。

综上，*Bowyer-Watson* 法通过逐点插入和局部重构实现 *Delaunay* 三角剖分。其优势在于每次插入新点时只处理受该点影响的局部区域，算法逻辑清晰，便于程序实现；同时，基于外接圆的判定机制能够保证最终三角网满足空圆准则，从而减少狭长三角形的出现，提高三角剖分在地形建模、空间插值和 TIN 构建中的几何稳定性。

用流程图表示如图2.3所示。

Bowyer-Watson 逐点插入法程序流程图

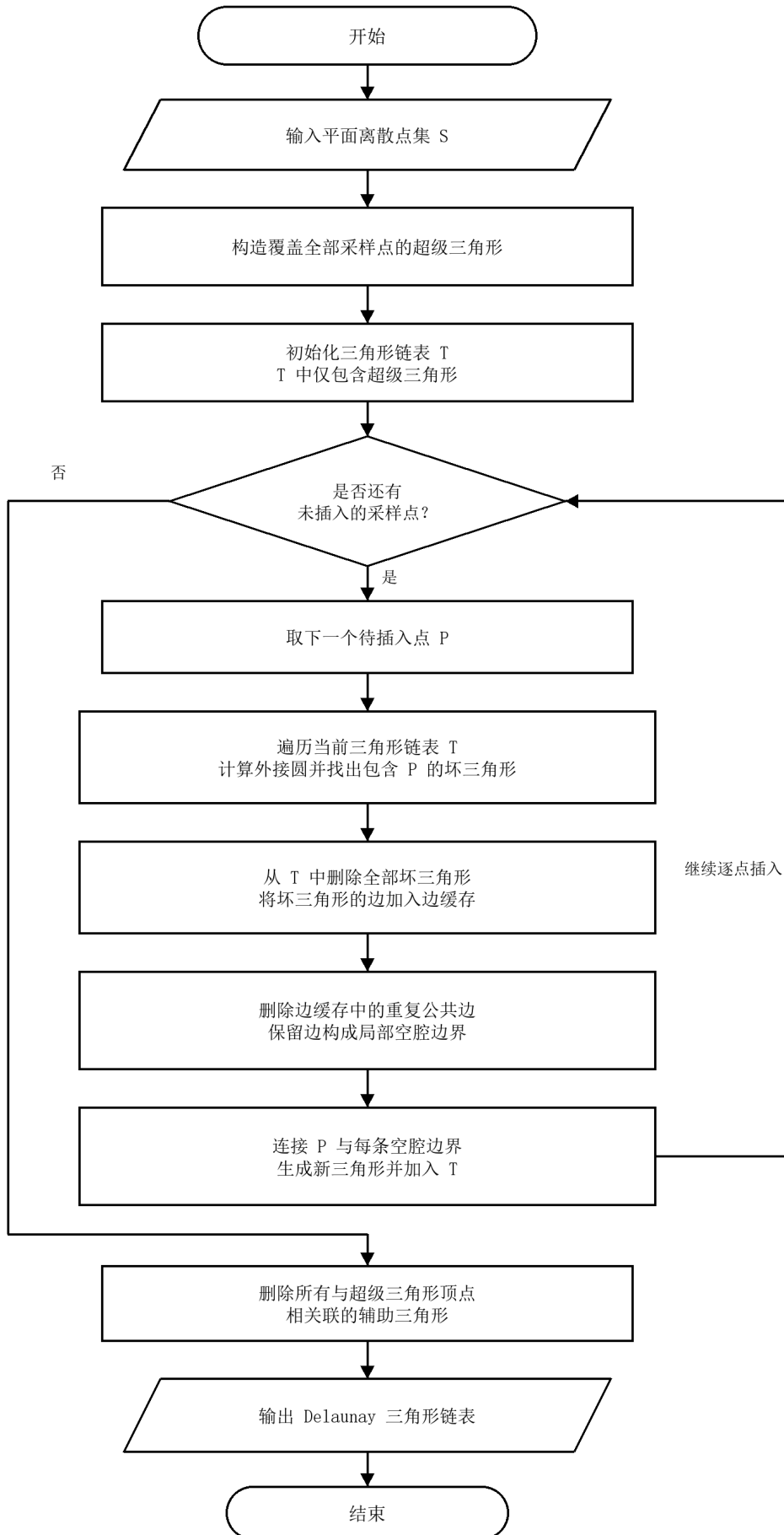


图2.3 Bowyer-Watsonz 逐点插入法

3.代码实现

3.1 开发环境

本项目在 Windows x64 平台下完成开发和测试，具体软硬件环境如下：

项目	配置信息
笔记本型号	天选 5 Pro
CPU	Intel(R) Core(TM) i9-14900HX
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
系统环境	Windows x64
开发软件	Visual Studio
编程语言	C++20

本程序主要使用 CPU 完成 *Delaunay* 三角剖分计算，GPU 不直接参与算法运算。Visual Studio 用于工程管理、代码编写、编译和调试，项目采用 x64 Console 配置，并使用 ISO C++20 标准。

3.2 程序模块

系统采用多文件模块化结构，将主程序、接口声明和算法实现分离。各文件的职责及依赖关系如下表所示。

文件/模块	类型	主要职责	接口与依赖关系
cppks.cpp	主程序源文件	定义 <code>main()</code> ，设置运行参数，调用构网、统计和导出流程	包含 <code>tin.h</code> ，通过接口调用算法与导出函数
tin.h	头文件/接口层	声明核心数据结构和函数原型，提供模块间共享的公共接口	被 <code>cppks.cpp</code> 和 <code>tin.cpp</code> 同时包含，是主程序与实现层之间的接口边界
tin.cpp	算法实现源文件	实现地形采样、外接圆计算、Bowyer-Watson 构网、边统计和文件导出	实现 <code>tin.h</code> 中声明的函数，并向输出文件写入结果
cppks.vcxproj	工程配置文件	配置 C++20、x64 Console 等编译目标，并组织源文件参与构建	统一管理 <code>cppks.cpp</code> 、 <code>tin.cpp</code> 和 <code>tin.h</code> 的工程编译关系

程序模块之间的关系可以表示为：

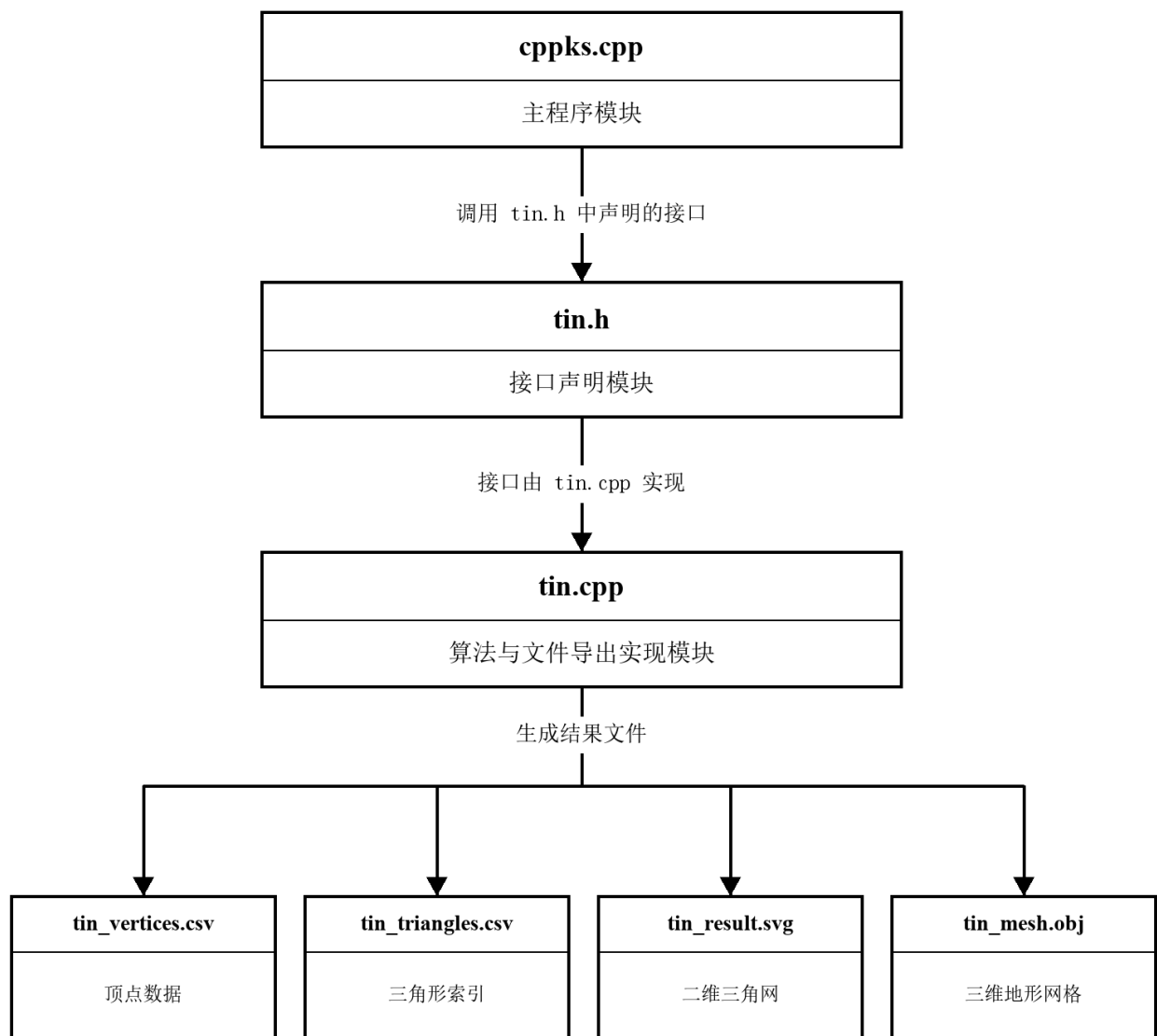


图3.1 程序模块之间关系

该组织方式将程序入口、公共接口和算法实现相互分离，降低了模块之间的耦合度，也符合实习任务对头文件声明和源文件实现的多文件结构要求。

3.3 主程序模块

主程序位于 `cppks.cpp` 中，负责组织程序的完整运行过程。

程序首先设置二维区域大小和采样点数量：

```
1 | constexpr int width = 640;
2 | constexpr int height = 480;
3 | constexpr int pointCount = 50;
```

其中，`width` 和 `height` 决定采样区域及 SVG 输出画布大小，`pointCount` 表示生成的地形采样点数量。

随后，程序依次完成采样点生成、*Delaunay* 三角剖分和网格边统计：

```

1  const std::vector<Point> points =
2      generateTerrainPoints(pointCount, width, height);
3
4  const std::vector<Triangle> triangles =
5      buildDelaunayTin(points);
6
7  const std::set<Edge> edges =
8      collectEdges(triangles);

```

构网完成后，主程序调用不同的输出函数，将同一份 TIN 数据导出为 CSV、SVG 和 OBJ 文件：

```

1  writeVerticesCsv(points, "tin_vertices.csv");
2  writeTrianglesCsv(triangles, "tin_triangles.csv");
3  writeSvg(points, triangles, "tin_result.svg", width, height);
4  writeObj(points, triangles, "tin_mesh.obj");

```

最后，程序输出节点数、三角形数、唯一边数、高程范围及生成的文件名。主程序不直接实现复杂的计算几何操作，只负责参数配置和功能调度，从而保持入口代码简洁。

3.4 数据结构与接口模块

核心数据结构和函数接口声明在 `tin.h` 中。程序定义了 `Point`、`Edge`、`Triangle` 和 `Circle` 四种结构。

```

1  struct Point {
2      double x;
3      double y;
4      double z;
5  };
6
7  struct Edge {
8      int a;
9      int b;
10 };
11
12 struct Triangle {
13     int a;
14     int b;
15     int c;
16 };
17
18 struct Circle {
19     double x;
20     double y;
21     double r2;
22     bool valid;
23 };

```

`Point` 用于保存地形采样点，其中 `x` 和 `y` 表示平面位置，`z` 表示高程。Delaunay 三角剖分只使用 `x` 和 `y` 构建平面拓扑关系，`z` 用于地形着色和三维模型表达。

`Edge` 使用两个顶点下标表示无向边。其构造函数始终将较小的顶点编号存入 `a`，较大的编号存入 `b`。因此，边 `(2,5)` 和边 `(5,2)` 会被统一表示为 `(2,5)`，便于使用 `std::map` 统计边的出现次数以及使用 `std::set` 去重。

`Triangle` 使用三个顶点下标表示一个三角面。程序通过二维叉积统一三角形顶点的逆时针排列顺序。

`Circle` 表示三角形外接圆，其中 `x`、`y` 是圆心坐标，`r2` 是半径平方，`valid` 表示外接圆是否有效。

3.5 地形采样点生成模块

地形采样点由 `generateTerrainPoints()` 生成。程序采用固定随机种子：

```
1 | std::mt19937 rng(20260531);
```

固定种子使每次运行生成相同的采样点，从而保证实验结果可重复，方便程序调试、结果验证和演示录制。

为防止采样点过度集中，程序要求任意两个点之间的平面距离不小于 22 个坐标单位。判断时使用距离平方： $(x_1 - x_2)^2 + (y_1 - y_2)^2 < 22^2$ 。使用距离平方可以避免执行平方根运算。

采样点通过间距检查后，程序调用 `terrainHeight()` 计算高程。地形函数由两个高斯山丘、一个高斯谷地和周期波叠加组成： $z = 80 + h_1 + h_2 - v + 10 \sin\left(\frac{x}{62}\right) + 7 \cos\left(\frac{y}{54}\right)$

该函数能够生成具有山峰、低谷和连续起伏的模拟地形，为后续 TIN 三维表达提供高程数据。

3.6 基础计算几何模块

程序使用二维叉积判断三个点的排列方向：

$$\text{cross}(A, B, C) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A)$$

当叉积大于 0 时，三个点按逆时针排列；当叉积小于 0 时，三个点按顺时针排列；当叉积为 0 时，三个点共线。

`makeCounterClockwise()` 检测三角形顶点方向。如果三角形为顺时针方向，则交换后两个顶点，使全部三角形保持统一的逆时针方向。统一顶点方向便于 OBJ 面片显示，也有利于后续扩展法向量、坡度和坡向计算。

`circumcircle()` 根据三个顶点计算三角形的外接圆。若计算公式的分母满足： $|d| < 10^{-12}$ ，则认为三个点近似共线，无法确定有限外接圆，此时返回无效圆。

程序保存外接圆半径的平方 r^2 ，并通过下式判断待插入点是否位于外接圆内部：

$$(x_P - x_O)^2 + (y_P - y_O)^2 \leq r^2 + \varepsilon$$

其中， $\varepsilon = 10^{-9}$ 是浮点容差。使用距离平方可以避免平方根计算，同时降低浮点误差对判断结果的影响。

3.7 Delaunay 三角剖分模块

Delaunay 三角剖分由 `buildDelaunayTin()` 实现，采用 Bowyer-Watson 逐点插入算法。

首先，程序计算全部采样点的平面包围盒，并在包围盒外构造一个足够大的超级三角形。超级三角形覆盖所有真实采样点，初始三角形集合中仅包含该三角形。

随后，程序按照输入顺序逐个插入采样点。对于当前待插入点，遍历已有三角形并进行外接圆判断：

```

1  if (pointInsideCircumcircle(
2      points[pointIndex], triangle, points)) {
3      ++edgeUseCount[Edge(triangle.a, triangle.b)];
4      ++edgeUseCount[Edge(triangle.b, triangle.c)];
5      ++edgeUseCount[Edge(triangle.c, triangle.a)];
6  }

```

若待插入点位于某个三角形的外接圆内部，则该三角形在加入新点后不再满足 Delaunay 空外接圆条件，因此被标记为坏三角形。

程序删除坏三角形，同时统计它们的边。相邻坏三角形之间的公共边会出现两次，而局部空腔边界只出现一次。因此，仅保留计数为 1 的边即可提取空腔边界。

```

1  for (const auto& [edge, useCount] : edgeUseCount) {
2      if (useCount == 1) {
3          kept.push_back(makeCounterClockwise(
4              {edge.a, edge.b, pointIndex}, points));
5      }
6  }

```

程序将待插入点与每条空腔边界连接，生成新的逆时针三角形，并将其加入三角形集合，从而完成局部三角网重构。

所有真实采样点插入完成后，程序删除包含超级三角形辅助顶点的三角面。剩余三角形即为最终的 Delaunay TIN。

3.8 网格统计模块

网格统计由 `collectEdges()` 实现。该函数遍历全部三角形，并将每个三角形的三条边插入 `std::set<Edge>`。由于 `Edge` 已统一端点顺序，因此相邻三角形共享的公共边只会保留一次。最终可以直接通过集合大小得到唯一边数。

平面三角网应满足欧拉关系： $V - E + F = 1$ 。其中， V 为节点数， E 为唯一边数， F 为三角面数。本次程序运行结果为： $V = 50$ ， $E = 138$ ， $F = 89$ ，代入后得到： $50 - 138 + 89 = 1$ 。结果满足平面三角网的欧拉关系，说明生成的三角网具有正确的拓扑结构。

3.9 高程颜色映射模块

`colorForHeight()` 用于将三角形平均高程映射为 SVG 填充颜色。程序首先将高程归一化到区间 $[0, 1]$ ：

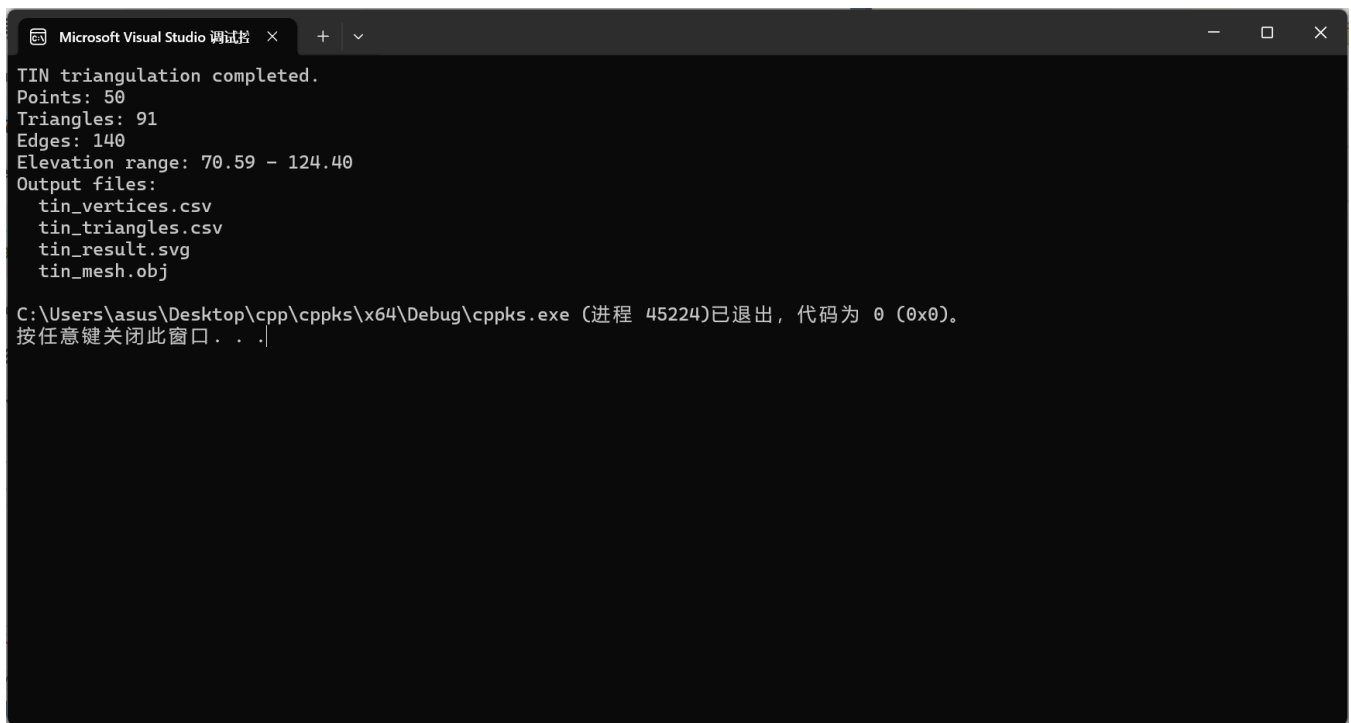
$$t = \frac{z - z_{\min}}{z_{\max} - z_{\min}}$$

在蓝、绿、黄、棕和浅灰白五种颜色之间进行线性插值。低高程区域使用蓝色和绿色，高高程区域使用黄色、棕色和浅色，从而形成直观的地形分层效果。

计算得到的 RGB 数值最终被转换为 SVG 使用的十六进制颜色：`#RRGGBB`。

3.10 结果导出模块

程序提供四种结果导出方式。



```
Microsoft Visual Studio 调试  x + v
TIN triangulation completed.
Points: 50
Triangles: 91
Edges: 140
Elevation range: 70.59 - 124.40
Output files:
  tin_vertices.csv
  tin_triangles.csv
  tin_result.svg
  tin_mesh.obj

C:\Users\asus\Desktop\cpp\cppks\x64\Debug\cppks.exe (进程 45224)已退出, 代码为 0 (0x0)。
按任意键关闭此窗口 . . .
```

图3.2 控制台输出

3.10.1 顶点数据导出

`writeVerticesCsv()` 将每个地形节点保存到 `tin_vertices.csv` :

```
1 | id,x,y,z
2 | 0,434.991,300.347,79.784
```

其中, `id` 是顶点编号, `x`、`y` 是平面坐标, `z` 是高程。

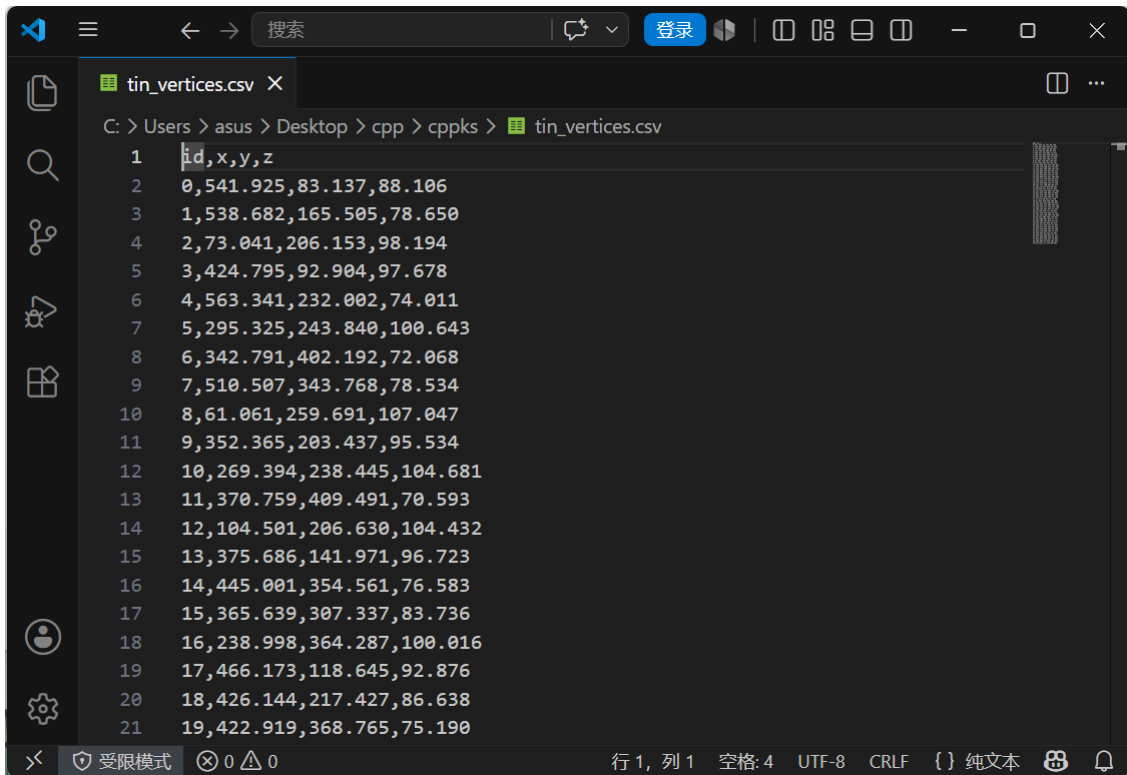


图3.3 tin_vertices.csv

3.10.2 三角形数据导出

`writeTrianglesCsv()` 将每个三角面的顶点索引保存到 `tin_triangles.csv` :

```
1 id,a,b,c
2 0,1,16,9
```

字段 `a`、`b`、`c` 对应顶点数据文件中的节点编号。

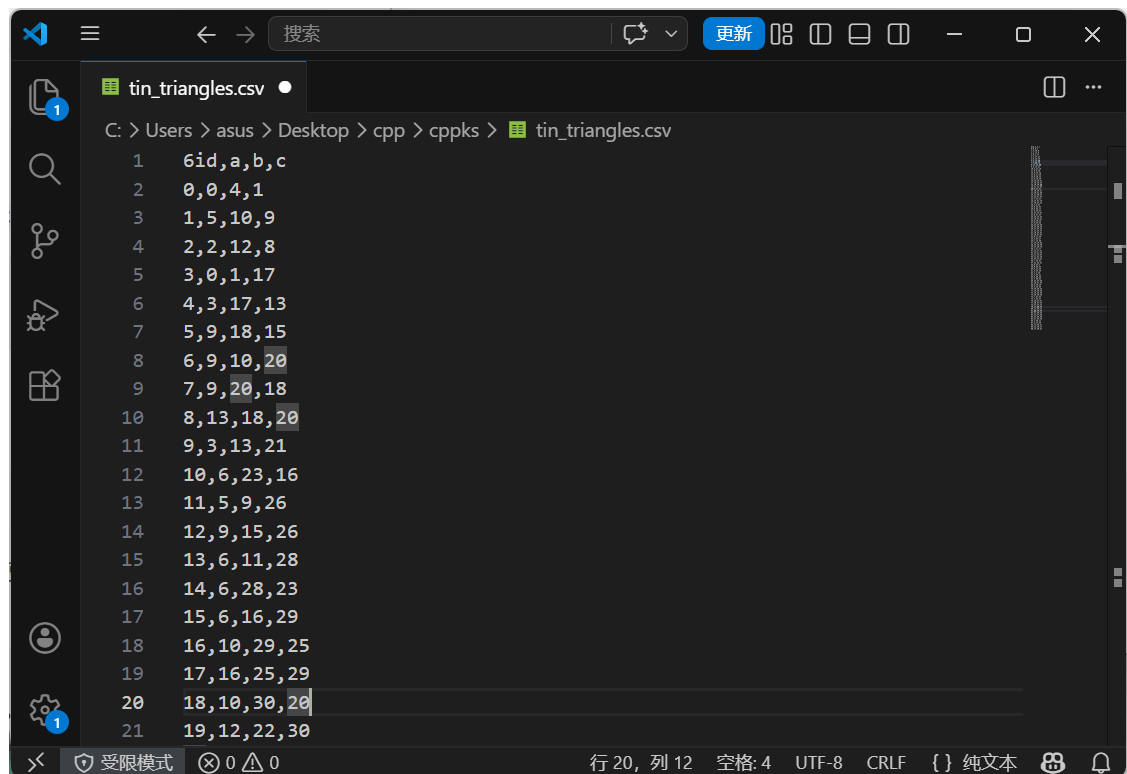


图3.4 tin_triangles.csv

3.10.3 二维 SVG 导出

`writeSvg()` 生成二维彩色 TIN 图，保存到 `tin_result.svg`。每个三角形使用三个顶点的平均高程确定填充颜色，同时绘制深色边界和黑色采样点。SVG 文件可以直接在浏览器中打开，便于观察三角形连接关系及地形高程分布。

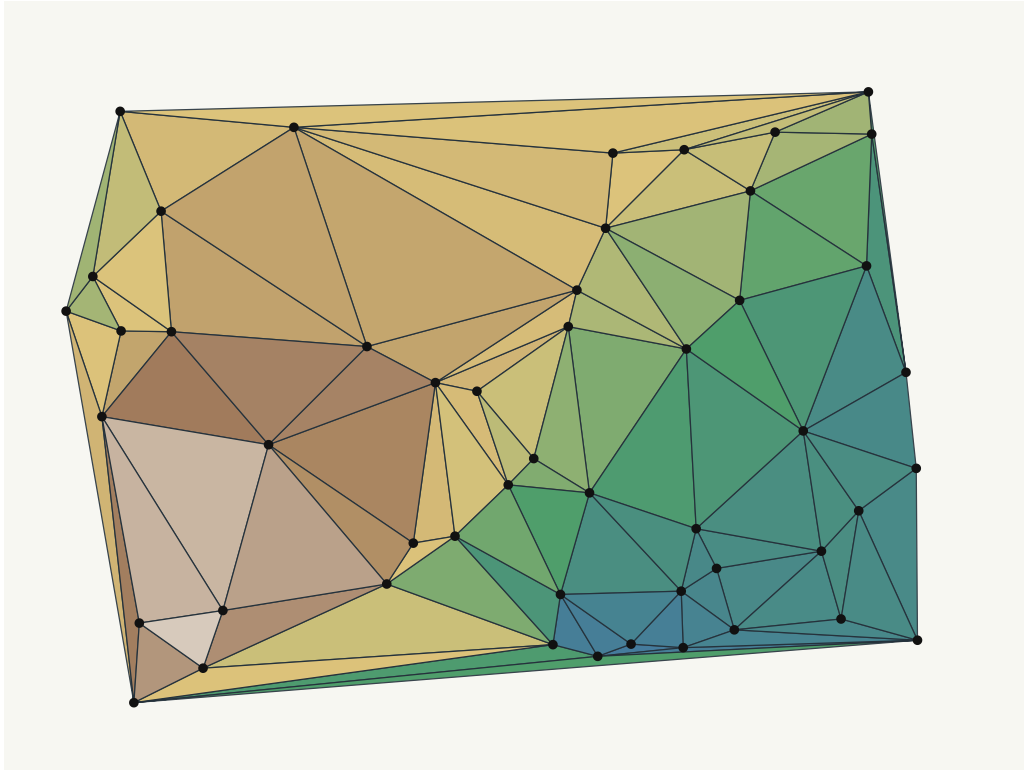


图3.5 tin_result.svg

3.10.4 三维 OBJ 导出

`writeObj()` 将 TIN 导出为三维网格模型。OBJ 文件使用 `v` 表示三维顶点，使用 `f` 表示三角面。程序按照 `v`、`x`、`z` 和 `y` 的顺序输出顶点，即将高程 `z` 放在三维软件的竖直坐标轴上。由于 OBJ 顶点索引从 1 开始，而 C++ 容器下标从 0 开始，因此输出三角面时需要将每个索引加 1。

生成的 `tin_mesh.obj` 可以导入 Blender 或 MeshLab，进行旋转、缩放和三维地形观察。

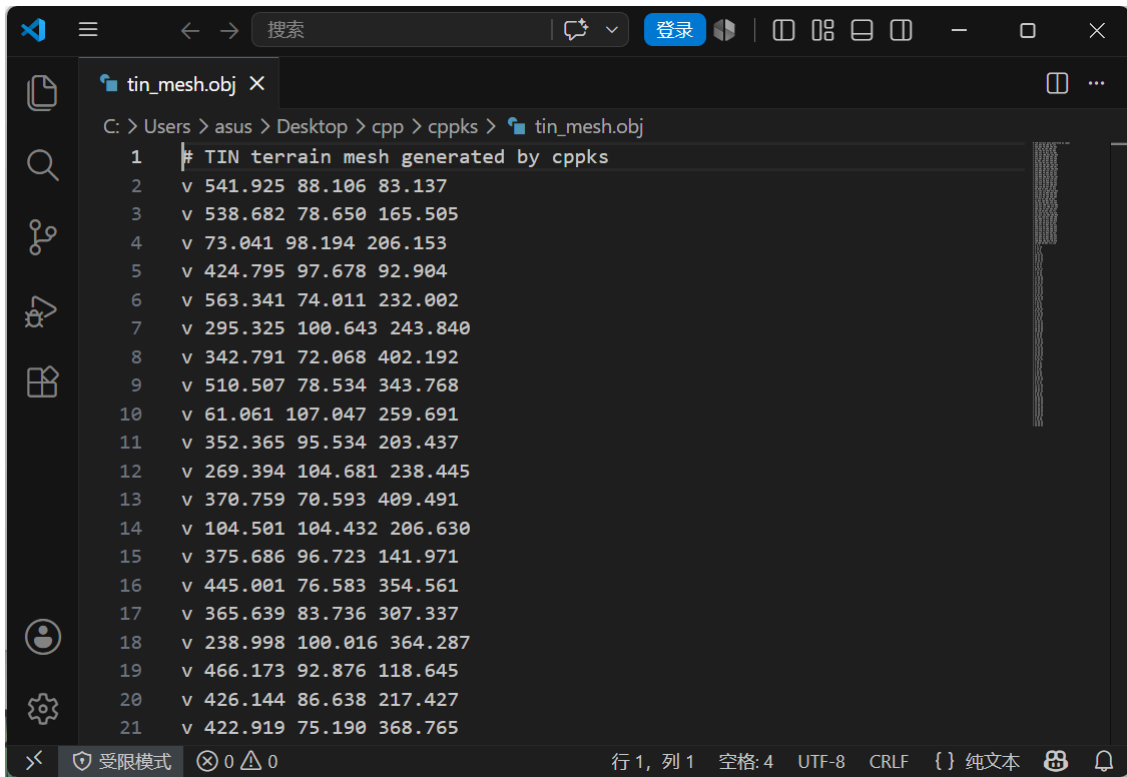


图3.6 tin_mesh.obj